

Sanha Maeng

RESEARCH ENGINEER AT MAUMAI

✉ sanha.maeng@gmail.com | 🏠 9rum.github.io | 🐙 [GitHub](https://github.com) | [LinkedIn](#) | 🎓 [Google Scholar](#)

Introduction

I am a research engineer at MaumAI Brain. My research interest lies in the duality of machine learning and computer systems, with a focus on systems for machine learning.

Before joining MaumAI, I received my M.S. in Computer Science and Engineering (CSE) from Sogang University, where I studied software systems for parallel training of neural networks.

At MaumAI, I have worked on efficient serving of generative models using hardware accelerators such as GPUs and NPUs. My work has covered a broad range of generative models, from GANs to LLMs and VLMs/VLAs, with a particular emphasis on Transformer-based generative models. I am currently focused on on-device serving of VLA models for physical intelligence – especially dual-system architectures on NPU-based edge devices.

Education

Sogang University

M.S. IN COMPUTER SCIENCE AND ENGINEERING

- Advisor: Prof. Sungyong Park

Seoul, Republic of Korea

Mar. 2021 - Feb. 2023

Kookmin University

B.S. IN COMPUTER SCIENCE AND ENGINEERING

Seoul, Republic of Korea

Mar. 2017 - Feb. 2021

Experience

MaumAI

RESEARCH ENGINEER (TECHNICAL RESEARCH PERSONNEL)

Seongnam, Republic of Korea

Mar. 2023 -

- Designed and implemented on-device inference engine named *Mona*, achieving **1.5×** faster TTFT and **36%** higher MMLU accuracy compared to Qualcomm GENIE.
- Achieved **0.6 seconds** TTFT latency in LLM serving on 12TOPS Qualcomm QCS6490.
- Improved LLM serving throughput by up to **352×** using continuous batching, PagedAttention and FlashAttention.
- Accelerated talking face generation models inference by 4× using FP16 quantization and TensorRT compilation, achieving up to **440×** speedup in time-to-first-frame latency with iteration-level scheduling. Further improved by **3×** with computation-I/O overlapping.

CONCAT

SOFTWARE ENGINEER INTERN

Seoul, Republic of Korea

Jun. 2020 - Sep. 2020

FlyHigh

SOFTWARE ENGINEER INTERN

Seongnam, Republic of Korea

Jun. 2019 - Aug. 2019

Projects

Speech-to-Face model inference optimization

MaumAI

DESIGN, OPTIMIZATION

Mar. 2023 - Jun. 2023

- The speech-to-face service uses talking face generation models.
- The original inference server had high latency due to its *request-level* scheduling policy.
- It responses after all frames have been generated, taking 1.1× response time compared to the input audio length.
- I introduced *iteration-level* scheduling along with FP16 quantization and compiled graph execution.
- As a result, the new inference server achieved up to **440×** speedup (110× by iteration-level scheduling, 2× by quantization, 2× by compilation), enabling real-time inference on T4 while maintaining device memory usage at 4GB.
- I observed that the next major bottleneck was the *synchronous* copy of generated frames to host in a blocking manner.
- The next version of inference server adopted computation-I/O overlapping via *asynchronous* copy, enabling **3×** maximum batch size.

LLM serving optimization and migration to vLLM

MaumAI

DESIGN, OPTIMIZATION, ENGINEERING

Jul. 2023 - May. 2024

- The former inference server suffered from high TTFT latency and low throughput, due to its *request-level* scheduling policy and *bucketing-based* execution mechanism.
- I adopted *iteration-level* scheduling and *FlashAttention* to reduce latency.
- While implementing *split attention* as an efficient batching technique for higher throughput, I found out an early version of vLLM and migrated the system to it, improving throughput by up to **352×**.
- To assess performance and determine the device specifications required to meet client demands, I developed a Poison process-based stress test generation tool.

On-device LLM inference optimization on Qualcomm QCS6490

MaumAI

DESIGN, OPTIMIZATION, TEAM LEAD

Jun. 2024 - Dec. 2024

- On-device deployment of LLM to QCS6490 is challenging, since the lack of VTCM sharing in Hexagon V68 architecture makes it difficult to parallelize the prefill phase while the NPU has only 12TOPS of INT8 performance.
- To overcome hardware constraints and satisfy the requirement for latency of < 1 second, we integrated the prefill and decode phases into a single phase by increasing the number of logits, enabling parallel prefill previously impossible on QCS6490.
- Such optimization comes with a performance trade-off between the prefill and decode phases, so we conducted an ablation study to find the optimal logit size that minimizes latency.
- As a result, we were able to reduce latency to **0.6 seconds** and successfully supply the inference engine for NAMUHX of SK Magic (currently SK intellix), a CES 2026 Innovation Award honoree.

Mona: Efficient VLA serving on Qualcomm SoCs

MaumAI

DESIGN, OPTIMIZATION, TEAM LEAD

Jan. 2025 - Dec. 2025

- Designed and implemented *Mona*, an inference engine for efficient serving of generative models on Qualcomm SoCs.
- The existing Qualcomm GENIE runtime repeatedly performed redundant computations and supported only naïve quantization. Since GENIE is closed-source, these limitations could not be addressed in our own hands.
- To overcome this, we developed *Mona* on top of ExecuTorch, an emerging open-source on-device inference engine.
- *Mona* incorporated *static prefix caching* and *SmoothQuant*, achieving **1.5×** faster TTFT than GENIE while improving MMLU accuracy by **36%** for the LLM previously deployed on QCS6490.
- For VLA serving on IQ9075, the model had to process long input sequences while satisfying 250ms end-to-end latency. The problem was further complicated by a hardware limitation that allowed only one of the two 50TOPS NPUs to be utilized, requiring significantly more aggressive optimization.
- To meet this target, we implemented *DivPrune* for visual token pruning in *Mona* and offloaded the GEMM operations required during pruning to Adreno GPU using OpenCL, reducing the pruning overhead by **40×** and overall inference latency by **4.56×**.

Dual-system optimization on Tenstorrent NPU

MaumAI

DESIGN, OPTIMIZATION, TEAM LEAD, SoC SELECTION

Jan. 2026 -

- After 1.5 years developing inference engine on top of QNN SDK, we found that further optimization is constrained by limited programmability. In particular, reducing excessive DRAM access through custom fused kernels was difficult because HMX instructions were not directly accessible.
- To overcome such limitation, I proposed a transition to Tenstorrent ecosystem, which provides a fully open software stack and control over the device.
- Since BOS semiconductors develops its on-device NPUs based on Tenstorrent IP, we ported and optimized a diverse set of generative models – including EXAONE, InternVL 3, LFM 2.5, Mamba, StableDiffusion and Bark – on A0 NPU.
- We are currently extending this work to support dual-system architecture, mapping both System 1 and System 2 onto Tenstorrent NPUs. Beyond this, I am exploring concurrent execution that enables both systems to run on a single NPU while maximizing hardware resource utilization.

(On-going research) Rethinking Throughput-Accuracy Trade-off in LLM Training

MaumAI

DESIGN, OPTIMIZATION, RESEARCH

Mar. 2026 -

- The *concat-and-chunk* scheme is widely-used for handling variable sequence lengths in LLM training, but introduces both throughput and accuracy degradation. It causes *cross-document attention* within sequences and weakens contextual coherence at sequence boundaries due to *document truncation*.
- Recent studies have addressed these issues independently but are not orthogonal, still force a trade-off between throughput and accuracy.
- I noticed this trade-off as a false dilemma and proposed *FlatFlow*, a training paradigm that aims to optimize both throughput and accuracy simultaneously rather than improving one at the expense of the other.
- In the offline stage, FlatFlow performs *document-wise chunking* to minimize document truncation. In the online stage, it adopts *split attention*, originally devised for serving, to improve throughput without cross-document attention.
- I found that naïvely applying split attention to training introduces several challenges. To address them, I proposed *concatenation-based batching* as a replacement for the canonical stack-based batching, since it is not suitable for use with split attention.
- I also introduced *iterative reordering* to reduce computation stalls caused by random sample ordering.
- Finally, because iterative reordering requires per-sample FLOPs that are not known a priori, I devised *symbolic transformation*, which decomposes and analyzes the computational graph obtained from model compilation to estimate the required FLOPs.

Publications

1. **Sanha Maeng**, Gordon Euhyun Moon and Sungyong Park, CHRONICA: A Data-Imbalance-Aware Scheduler for Distributed Deep Learning, 23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid '23) paper [🔗](#)

Open Source Contributions

1. Contributed to ExecuTorch regarding Qualcomm QCM6490 support, becoming one of the ExecuTorch v1.1.0 contributors [pytorch#16331](#) [🔗](#) [release note](#) [🔗](#)
2. Contributed to Amazon Deep Graph Library (DGL) regarding distributed job launcher, becoming one of the DGL v2.0.0 contributors [dmlc#6304](#) [🔗](#) [release note](#) [🔗](#)

Honors and Awards

- 2022 **Best Paper Award**, Korea Software Congress
- 2020 **2nd Place**, KMUCS Capstone Design Conference

KIISE

KMU